Uniform binding for intuitionistic primitive recursive arithmetic

A smaller complete checker, with an explicit bi-interpretation

Research note prepared for Vladimir Reshetnikov

11 November 2026

Repository snapshot: `0ebd0c6b358f38bb0ad2883faf775421599c8aae`

Abstract

A modest simplification of the current ConciseIPRA presentation is obtained by combining its counter-aware recursor, its Eq/D clause calculus, and a single syntax traversal that performs validation, canonicalization, and binder opening. The numerical recursor and the logical binder each bind two variables in their final child. There is no independent successor constructor, equality template, closure-formula constructor, or general computation normalizer. The resulting complete Python checker has **2,238 normalized bytes, 693 significant tokens, and 764 AST nodes**, compared with 2,283, 741, and 826 for the strongest comparable incoming scalar checker inspected. The three reductions are respectively 1.97%, 6.48%, and 7.51%. Maximum-compression gzip grows from 818 to 866 bytes; it is reported without being used as the optimization target.

The arithmetic recursor, five-rule structure, and equality bootstrap are inherited results, not new discoveries. The contribution is a smaller complete realization of their combination, with a matched control and portable verification. The report states every primitive rule and proves the full first-order iPRA comparison, including both interpretation composites. Both numerical domains are whole, equality is actual equality, and the composite isomorphisms have graph $x = y$. This is neither an absolute-minimality theorem nor a new Lean verification.

1 Result, comparison, and the precise claim

The task is to simplify the current proposal, not an earlier version of the scalar kernel. Accordingly the comparison was pinned to the ConciseIPRA subtree at commit `0ebd0c6b358f38bb0ad2883faf775421599c8aae`. The repository README and all six incoming archives present in that snapshot were inspected. In particular, the incoming `clausal_single_evaluation` package already has a five-pattern goal-directed checker with one evaluation schema [1, 3]. Repeating its rule inventory would not advance the proposal.

The useful remaining change is to align the two binding signatures. For the scalar iterator, the computational body binds one variable but D binds two. The counter-aware recursor binds two. A single sorted tree traversal can therefore implement the syntax invariant and all opening operations, with one binding-depth rule. The clause context also determines the number of available free parameters, so the checker need not pass a separate scope counter.

Theorem 1.1 (Main mathematical claim)**.** *The explicitly specified canonical clause calculus $\mathcal{C}_R$ below is equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. Its ordinary realization uses whole numerical domains and actual equality. Each numerical composite interpretation is provably the identity, with defining graph $x = y$. The logical formula composites are provably equivalent to the original formulas.*

Theorem 1.1 is a written mathematical theorem. The implementation is independently tested, but neither those tests nor the existing Lean development are described as a new formal verification of this checker.

1.1 Complete-module measurements

All columns count the complete standalone module: imports, input handling, syntax and scope validation, canonicalization, substitutions, proof rules, and command-line behavior. The normalization convention removes

the true module docstring, runs CPython 3.12.14 `ast.unparse`, and appends one line feed. Significant tokens exclude whitespace, indentation, comments, and the end marker. AST nodes count the parsed normalized module.

Complete checker	Bytes	Tokens	AST	Gzip-9
Incoming scalar, exact source	2283	741	826	818
Scalar, matched shared traversal	2284	699	773	878
Uniform-binding R, selected	**2238**	**693**	**764**	866
Inherited forward counter checker	3004	854	987	1001

The matched scalar control uses the same goal-directed organization, shared traversal, and inferred scope. Against it, the counter-aware representation saves 46 bytes, 6 tokens, and 9 AST nodes. Against the exact incoming checker it saves 45 bytes, 48 tokens, and 62 AST nodes. This separates the representation effect from routine implementation consolidation. No claim rests on comparing a readable implementation with a partially counted core.

The original formatted selected file is 2,694 bytes, versus 2,717 for the exact incoming scalar file. Their raw-source gzip-9 sizes are 1,124 and 1,036 bytes. Both normalized and original-source compressed sizes are included in the artifacts, with compression level 9 and timestamp zero.

The repository also contains much shorter compact protocols, including a reported 516-byte codec and an incoming 820-byte native validator. They are different complete encodings/interfaces; the present result does *not* establish a repository-wide size minimum. In particular, the word *smallest* is not assigned an invariant mathematical meaning by a rule count or one chosen programming language.

1.2 What is inherited and what changes

The successor-seeded counter recursor and its one-rule equality mechanism come from `counter_evaluation`; clause contexts and the goal-directed single-evaluation organization come from the current clause packages [2, 3, 4]. Inferring scope from the context is also present in an incoming proposal [5]. The upstream counter checker already exploits equal binding arities in its opening function. The additional consolidation here is to make the *same sorted traversal* own grammar validation, canonical-form checking, and opening, and to measure the resulting complete goal checker against both the exact incoming program and a matched scalar control.

There is no newly deleted arithmetic principle, and no claim of priority outside the inspected snapshot. This is a measured improvement in the specified proof-checker sense, with the mathematical dependency boundary made explicit.

2 Language and canonical substitution

There is one numerical sort. Raw terms are

$$t ::= x \mid 0 \mid R(n, a; i, u.h).$$

Only i, u are bound in h; neither is bound in n or a. The counter variable is written first and the current-state variable second. Successor and one are abbreviations:

$$S(a) := R(0, a; i, u.0), \qquad 1 := S0.$$

Thus successor remains an arithmetic operation, but it is not a separate term constructor.

Canonical terms require a recursor with *literal* counter 0 to have literal body 0. Let N recursively canonicalize children and then replace

$$R(0, a; i, u.h) \quad \text{by} \quad R(0, \mathsf{N}(a); i, u.0).$$

All substitutions below are capture avoiding and followed by N. The body is still a syntactic object which must be checked before its erasure; malformed proof data cannot acquire legitimacy merely because its value would be ignored. No successor computation is performed by N, and no proof search or semantic equality test is hidden in it.

Formulas are

$$A ::= (s = t) \mid D_{x,y}B.$$

Both x, y are bound in the entire matrix B. The intended ordinary reading is

$$(D_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big). \tag{D}$$

The designated contradiction is $\bot_* = (0 = 1)$; the convenient neutral truth is $H = (0 = 0)$.

A reusable clause is a pair

$$(A \Rightarrow_y B(y)).$$

Its ordinary reading is $A^\circ \to \forall y\, B^\circ(y)$, with y absent from A. A context Γ is a finite list of such clauses. An ordinary local hypothesis A is represented by the clause $H \Rightarrow_\nu A$, where ν is unused. No primitive formula Q, list of logical axioms, or separate equality-template syntax is present.

2.1 The one binding-depth rule

The concrete trees have arities

$$\underbrace{[B]}_{D,\ \text{arity } 1}, \qquad \underbrace{[s,t]}_{\text{equation, arity } 2}, \qquad \underbrace{[n,a,h]}_{R,\ \text{arity } 3}.$$

Every non-equational constructor binds exactly two variables in its *last* child. Thus, for child number j in a node of arity k, the depth increment is

$$2[\,k \neq 2 \text{ and } j = k-1\,]. \tag{Uniform binding}$$

The term/formula sort remains checked. Formula arities are 1 and 2; the only nonvariable term arity is 3. The arity convention does not identify a formula with a numerical term.

Lemma 2.1 (Elementary canonical coherence)**.** *Canonicalization terminates, is idempotent and scope preserving, and obeys*

$$\mathsf{N}(t\sigma) = \mathsf{N}(\mathsf{N}(t)\mathsf{N}(\sigma))$$

for capture-avoiding numerical substitution; the same holds for formulas.

Proof. It is a finite-tree traversal. Induct on the input tree, lifting the substitution under each binder. The only exceptional node is a recursor whose counter becomes literal zero. Both sides then erase its body. If an earlier traversal already erased that body, its counter is literal zero and remains zero under every further substitution. All other nodes follow from their children. The argument is syntactic and uses no arithmetic induction or equality axiom. □

3 Every primitive rule

All terms and formulas in the rules are canonical. The two-variable substitution in CE is simultaneous. Names used as fresh parameters are new for the displayed ambient context, conclusion, and witness data.

Reflexivity.

$$\frac{}{\Gamma \vdash t = t}\quad (\mathsf{Refl}).$$

Counter evaluation.

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash R(Ss, a; i, u.h) = r}{\Gamma \vdash r = h[t/i, R(t, a; i, u.h)/u]}\quad (\mathsf{CE}). \tag{CE}$$

The seed and body are fixed by the second premise. Passive occurrences of s inside that seed or body are not replaced. There are no other computation axioms or primitive equality inferences.

Use of a retained clause.

$$\frac{(A \Rightarrow_y B(y)) \in \Gamma \qquad \Gamma \vdash A}{\Gamma \vdash B(t)}\quad (\mathsf{Use}).$$

The target term t is any term in the current numerical scope. This rule includes use of an ordinary assumption, since H is reflexive.

Opening a paired binder.

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, \big(B(b,0) \Rightarrow_y B(b,y)\big) \vdash A}{\Gamma \vdash A}\ (\mathsf{Open}), \qquad b \text{ fresh.}$$

Closing a paired binder. Put

$$\eta = \begin{cases} z, & \text{if } B \text{ is literally an equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

Then

$$\frac{\Gamma, \big(H \Rightarrow_\nu B(t,\eta)\big) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B}\ (\mathsf{Close}), \qquad z \text{ fresh}, \quad \nu \text{ unused.} \tag{Close}$$

Fresh z may occur in the local clause when $\eta = z$. This is the equational induction branch of the rule. The other branch is numerical case introduction. The rule's syntactic test is part of its specification; it is not asserted to commute with arbitrary substitution by a naive induction on proofs.

These five schemes, the canonical syntax convention, and the empty initial context are the complete presentation. Weakening, cut, substitution, full formula transport, ordinary intuitionistic logic, successor injectivity, coverage, and induction for quantifier-free formulas are derived. Their admissibility is justified below, not silently added to the checker.

4 Arithmetic from one evaluation rule

4.1 The counter-evaluation equality rule

Reflexivity and the single counter-evaluation rule are

$$\frac{}{t = t}, \qquad \frac{s = t \qquad R(Ss, a; i, u.h) = r}{r = h[t/i, R(t, a; i, u.h)/u]}. \tag{CE}$$

Every term in this display denotes its canonical form, and body substitutions are simultaneous and capture avoiding. In particular the second occurrence of R becomes Sa when t is literally zero. The seed and update body are read from the second premise and are not rewritten at passive occurrences of s.

Two applications recover common-left equality composition. Put $K_s = R(Ss, 0; i, u.i)$. Given $p : s = t$ and $q : s = r$, apply CE first to q and $K_s = K_s$, obtaining $K_s = r$, and then to p and that result, obtaining $r = t$. Therefore symmetry follows from $s = s$ and $s = t$, and transitivity follows by reversing the first premise and joining at its common left endpoint. No induction or logical operation is used.

For any fixed numerical template $f(v)$, put $L_s = R(Ss, 0; i, u.f(i))$. CE on $s = s$ and $L_s = L_s$ gives $L_s = f(s)$. Applying CE again with $s = t$ gives $f(s) = f(t)$. This derives unrestricted term congruence, including repeated occurrences and occurrences under nested recursor binders. Joining this equation with $f(s) = r$ derives crossed transport $r = f(t)$. Finally, CE on the two appropriate reflexivities is exactly the successor computation equation stated by CE.

4.2 Predecessor and every ordinary primitive recursion

Define the following finite terms, with all auxiliary binders fresh:

$$\begin{aligned} P(n) &= R(n, 0; i, u.i), \\ C(n, a, b) &= P(R(n, a; i, u.Sb)), \\ \mathrm{Rec}(n, a; i, u.h) &= P\big(R(n, a; i, v.S(h(i, P(v))))\big). \end{aligned}$$

Computation and congruence alone prove

$$\begin{aligned} P(0) &= 1, & P(Sn) &= n, \\ C(0, a, b) &= a, & C(Sn, a, b) &= b, \\ \mathrm{Rec}(0, a; h) &= a, & \mathrm{Rec}(Sn, a; h) &= h(n, \mathrm{Rec}(n, a; h)). \end{aligned} \tag{Compiler equations}$$

Indeed the inner trajectory $T(n) = R(n, a; i, v.S(h(i, P(v))))$ has $T(0) = Sa$ and $T(Sn) = S(h(n, P(T(n))))$. Applying $P(Sx) = x$ gives both equations. This calculation explains why the successor seed does not restrict allowed initial states: the state is stored as a successor, and the outer P removes exactly that wrapper after each observation. No informal claim about a trajectory's values substitutes for the displayed equations.

The term P is not the usual predecessor at zero. Define instead

$$p(n) = C(n, 0, P(n)), \qquad \mathrm{sg}(n) = C(n, 0, 1).$$

Then $p(0) = 0$, $p(Sn) = n$, $\mathrm{sg}(0) = 0$, and $\mathrm{sg}(Sn) = 1$. Applying P to $Sx = Sy$ proves successor injectivity. Applying sg to $Sx = 0$ gives $1 = 0$, contradicting the ordinary-realization separation axiom $0 = 1 \to \bot$.

Scalar iteration is the special case

$$I(n, a; u.h) = \mathrm{Rec}(n, a; i, u.h), \qquad i \notin \mathrm{FV}(h).$$

It has $I(0, a; h) = a$ and $I(Sn, a; h) = h(I(n, a; h))$ for every seed a. This recovers either raw scalar iteration up to provable equality or its zero-normal version under normalizing substitution. The counter-aware language can compile general primitive recursion directly: zero, successor, and projections have their evident finite terms; composition is substitution; recursion is the displayed Rec construction. There is no need to construct pairing before compiling recursion. This is an external compiler for each fixed primitive-recursive description, not an internally total universal evaluator for description codes.

5 Why the five rules recover full intuitionistic logic

The clause method, gate construction, and logical completeness argument in this section are inherited from the ConciseIPRA developments [6, 2, 3]. We give the details for the present counter-aware terms, including the witness-sensitive introduction. The immediate predecessor $P(Sn) = n$ also shortens the conjunction and universal macros: no parity iterator is needed. All statements below concern arbitrary retained contexts.

5.1 The clause interface and its first admissible operations

Formulas are $F ::= (s = t) \mid D_{x,y}F$. A context is a reusable list of clause occurrences $[A; y.B(y)]$, where y is absent from A. Put $H = (0 = 0)$. An ordinary assumption F means the constant clause $[H; v.F]$, with v unused. In addition to Refl and counter evaluation CE, the logical instructions are exactly

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \text{Use} \quad ([A; y.B] \in \Gamma),$$

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \quad \Gamma, [B(b, 0); y.B(b, y)] \vdash F}{\Gamma \vdash F}\ \text{Open},$$

$$\frac{\Gamma, B(t, \eta) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}\ \text{Close}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases}$$

Here b in Open is fresh for the retained context, conclusion, and passive matrix parameters. In Close, z is fresh for the context, witness, and passive matrix parameters. All substitutions are capture avoiding and canonical; $\equiv$ tests literal syntax. An equation under a D is not a literal equation. Local assumptions and opened clauses are discharged.

Refl proves H, so Use gives ordinary assumption use. Weakening, injective fresh renaming, and ordinary-assumption cut follow directly by induction on finite derivations. For cut, replace every Use selecting the distinguished constant clause $[H; v.F]$ by the supplied proof of F; the proof of its antecedent H can be discarded. Freshen eigenparameters before inserting the proof. These operations preserve the literal-zero witness test. Arbitrary numerical substitution requires the repair below; it is not assumed at this stage.

5.2 Cases and induction, before compound formula transport

The arithmetic equality lemma from Refl and CE gives transport in every numerical template and hence every equation template. Put

$$Z = R(S0, 0; i, u.0).$$

Its syntax is not the literal zero, while successor computation proves $Z = 0$. Suppose an equation matrix E has a case branch $\Gamma, E(0,0) \vdash E(0, Sz)$. Under the assumption $E(Z, 0)$, equation transport gives $E(0,0)$; cut inserts the supplied branch; reverse transport gives $E(Z, Sz)$. Close at witness Z is in case mode and concludes $D_{x,y}E(x,y)$. This proves every missing equation-case instance at witness zero. Every other case instance was already primitive. Thus

$$\frac{\Gamma, B(t,0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x,y)} \quad \text{(Case)} \qquad \text{(Case)}$$

is admissible for every matrix and every witness. Its repair used only atomic equality transport, not transport through a binder.

For an unused coordinate u, abbreviate $D_{u,y}A(y)$ temporarily by q_yA. Case introduces it from $A(0) \vdash A(Sz)$. Open and Use specialize it: from q_yA and $A(0)$ derive $A(t)$. Consequently the following two derived rules hold:

$$\frac{\Gamma \vdash F(0) \quad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \quad \text{(numerical cases)}, \qquad \frac{\Gamma \vdash E(0) \quad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \quad \text{(equation induction)}. \quad \text{(NC, EI)}$$

In the first rule, Case introduces q_yF while ignoring its assumed base. In the second, Close at unused witness 0 introduces q_yE directly in progress mode. In both cases Open supplies the conditional clause, the given base supplies its antecedent, and Use at t finishes. The displayed z is fresh for the retained context and passive parameters. Only E is required to be a literal equation.

Capture-avoiding numerical substitution is now admissible by proof induction. A progress introduction keeps its equation root and literal-zero witness. A case introduction whose witness becomes zero is replaced by the derived Case rule. Refl, CE, Use, and Open commute with canonical substitution, using Lemma 2.1 and freshening eigenparameters first. This justifies the substitutions of derivations used below without assuming witness-test invariance.

5.3 Transport, gates, and selection

Full formula transport is proved by external induction on $F(v)$. Equations were already handled. For $F(v) = D_{x,y}B(v,x,y)$, open $F(s)$ at fresh b, retaining $[B(s,b,0); y.B(s,b,y)]$. Introduce $F(t)$ by Case at witness b. Convert the local assumption $B(t,b,0)$ backwards along $s = t$ using the smaller-template induction hypothesis; Use the old clause at Sz; convert its result forwards to $B(t,b,Sz)$. Close Case and Open. This proves

$$s = t, \ F(s) \vdash F(t).$$

The same construction proves binder congruence for uniform pointwise interderivability: convert the target base backwards, specialize the old clause, and convert its successor instance forwards. It uses Case and cut, not induction on a compound arithmetic formula.

The previously computed selector satisfies $C(0,a,b) = a$ and $C(Sz,a,b) = b$. Define gates on finite formula syntax by

$$G_v(s = t) = (C(v,0,s) = C(v,0,t)), \qquad G_v(D_{x,y}B) = D_{x,y}G_vB,$$

with binders renamed away from v. Atomic equality and binder congruence prove, by external formula induction,

$$\vdash G_0F, \qquad G_{Sz}F \dashv\vdash F.$$

For the first statement at a paired formula, use Case and the already proved gate theorem for its smaller matrix; the assumed base is unused.

Padding $L(F) = D_{u,y}G_yF$, with fresh unused u, y, is interderivable with F. Forward, Case uses F to prove the successor gate. Backward, Open, the zero-gate theorem, and Use at 1 give G_1F, hence F. Every formula is a finite chain of paired binders ending in an equation. Pad the shorter of A, B to equalize their depths and align bound names. At their atomic leaves set

$$M_v((s = t), (a = b)) = (C(v,s,a) = C(v,t,b))$$

and carry the matched binders outside. Thus

$$M_0(A,B) \dashv\vdash A, \qquad M_{Sz}(A,B) \dashv\vdash B. \tag{Selector endpoints}$$

All these are finite macros in the original Eq/D language.

5.4 Every ordinary intuitionistic connective and quantifier

With all auxiliary binders fresh, and u unused, define

$$\begin{aligned} \bot_* &= (0=1),\\ A \to_* B &= D_{u,y} M_y(A,B),\\ A \wedge_* B &= D_{u,y} G_y M_{P(y)}(A,B),\\ \exists_* x\, A(x) &= D_{x,y} G_y A(x),\\ A \vee_* B &= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &= D_{u,y} G_y A(P(y)). \end{aligned} \tag{Logical macros}$$

Here $P(Sz) = z$, so $P(1) = 0$ and $P(2) = 1$. Its value $P(0) = 1$ causes no problem: the outer gate makes the zero instance provable.

Implication. Given a derivation of B under ordinary assumption A, Case introduces $A \to_* B$: the selector's zero instance gives A, and its successor instance is equivalent to B. For elimination, Open retains the selector clause; the supplied A gives its zero antecedent and Use at 1 gives B. Discharge and eigenvariable conditions are exactly the ordinary ones.

Conjunction. From A and B, numerical cases prove $M_v(A,B)$ for arbitrary v. Substitute $v = P(Sz)$, apply the successor gate, and use Case to introduce $A \wedge_* B$. Its zero antecedent is a gate theorem. For elimination, Open and Use at 1 and 2 give the selector at $P(1) = 0$ and $P(2) = 1$, hence A and B. No parity or range theorem is used.

Existential quantification. From $A(t)$, Case at witness t introduces its gated paired formula. Conversely Open at fresh b, prove the zero-gate antecedent, and Use at 1 to obtain $A(b)$. Cut this into any supplied branch whose conclusion and retained context are free of b, then close Open.

Disjunction. Choose witness 0 or 1 in the existential macro to obtain the two injections. For elimination, first discharge the two supplied branches as $A \to_* F$ and $B \to_* F$. Numerical cases, in the original context, prove $M_v(A,B) \to_* F$ for arbitrary v, using the selector endpoints. Existential elimination supplies $M_b(A,B)$ at a fresh tag b; implication elimination gives F. This construction does not substitute the tag inside a retained assumption: numerical cases were performed before that assumption was added.

Universal quantification. For introduction, a derivation of $A(z)$ with z fresh for the context gives the Case branch $G_{Sz}A(P(Sz))$, by $P(Sz) = z$ and the gate equivalence. Introduce the unused witness to obtain $\forall_* x A(x)$. For elimination, Open its clause and prove its zero-gate antecedent. Use at St gives $G_{St}A(P(St))$, hence $A(t)$. Thus every numerical target is covered directly; no claim about $P(0)$ is needed.

Falsity and numerical coverage. Under $0 = 1$, transport through the term $C(v,s,t)$ gives every equation $s = t$ from its two selector computations. External induction on formula depth, using Case, gives every Eq/D formula. This is ex falso. Numerical cases on $n = 0 \vee_* \exists_* m\,(n = Sm)$ prove coverage: use the left injection at zero, and the right injection with witness z at Sz. Coverage can then be used in contexts already containing n, through ordinary disjunction elimination and equality transport.

5.5 Faithful ordinary realization and both formula composites

Let T_R be ordinary intuitionistic first-order logic with equality over *raw* R terms, coverage, separation $0 = 1 \to \bot$, contextual literal-equation induction, and both computation schemas

$$R(0, a; i, u.h) = Sa, \qquad R(Sn, a; i, u.h) = h[n/i, R(n, a; i, u.h)/u].$$

Canonical native terms are read as these raw terms. The semantic justification of canonical substitution uses the ordinary theory's body-extensionality argument in Section 6.3; it is separate from the elementary syntactic coherence lemma. Read

$$(D_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)), \qquad [A; y.B]^\circ = A^\circ \to \forall y B^\circ(y).$$

Refl and CE are sound by the arithmetic equality and canonical-substitution lemmas. Use and Open are ordinary implication/universal and existential elimination. Close in case mode is sound by coverage. In progress mode, assume $E(0, 0)$ and use equation induction with its premise to prove $\forall y E(0, y)$, then choose witness 0. Thus every primitive native derivation has its ordinary realization in T_R.

Conversely the derived logical rules above translate every ordinary intuitionistic inference. Numerical equality and both computations are derived from Refl and CE; coverage and equation induction were derived above. The separation axiom translates to the immediate implication $(0 = 1) \to_* (0 = 1)$. Hence all T_R derivations translate into the native calculus. The ordinary reading of each displayed macro is provably equivalent to its ordinary connective: use the gate and selector endpoints, coverage for arbitrary controls, and $P(St) = t$ for the universal macro. External induction on ordinary formulas proves this formula round trip.

For the other composite, externally induct on a native formula. Equations are unchanged. For $D_{x,y}B$, first use the induction hypothesis for its matrix uniformly. From the original paired formula, Open at b. Under its base assumption, Use its clause at a fresh z to obtain $B(b, z)$; universal introduction, implication introduction, and existential introduction then build the macro translation of $\exists x(B(x, 0) \to \forall y B(x, y))$. Close Open. Conversely, existential elimination on that macro gives a fresh b and the implication from $B(b, 0)$ to its universal closure. Introduce the original paired formula by Case at b: its local base feeds the implication, and universal elimination at Sz supplies the required successor instance. The matrix induction equivalences convert the occurrences in both directions. This proves native interderivability with its round trip under arbitrary ordinary side assumptions. Both numerical maps in this logical comparison are identities; no extra domain or equality quotient is introduced.

The separation clause in the ordinary realization is explicit. Merely collecting the positive Eq/D theorem images would admit singleton models and would not be the asserted ordinary theory. The argument establishes the full first-order consequence relation; it neither postulates excluded middle for quantified formulas nor introduces induction for compound native formulas.

6 The direct iPRA bi-interpretation

6.1 The equation-induction to iPRA step

At this point the logical interface supplies ordinary intuitionistic logic, coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, separation, and induction for literal equations with arbitrary retained side assumptions. These are exactly the assumptions used in the following internal arithmetic argument.

Using Rec, define addition by $x + 0 = x$ and $x + Sy = S(x + y)$, and truncated subtraction by $d(x, 0) = x$, $d(x, Sy) = p(d(x, y))$. Equation induction proves $0 + y = y$ and $Sx + y = S(x + y)$. Equation induction in y in

$$p(d(Sx, y)) = d(x, y)$$

has base $p(Sx) = x$ and step obtained by applying p to its hypothesis. It gives $d(Sx, Sy) = d(x, y)$, hence a further equation induction gives $d(Sa, a) = 1$.

The remainder equation

$$C(\mathrm{sg}(d(x, y)), x, y + d(x, y)) = x \tag{Rem}$$

also follows by equation induction on y, keeping x fixed. At zero, both selector alternatives equal x. At the step put $r = d(x, y)$ and $r' = p(r)$. Coverage splits $r' = 0$ from $r' = Sz$. In the zero case the new equation selects

$x = x$. In the successor case, coverage of r, the predecessor equations, and separation imply $r = Sr'$. The old equation then gives $y + r = x$, and the new selected sum is

$$Sy + r' = S(y + r') = y + Sr' = y + r = x.$$

The branch distinctions use coverage, not induction on a disjunction. The only induction hypothesis throughout is the single equation (Rem).

If $d(Sa, b) = 1$, (Rem) gives $b + 1 = Sa$ and hence $a = b$ by successor injectivity. The reverse implication uses $d(Sa, a) = 1$. Put

$$K(n) = C(n, 1, p(n)), \qquad \epsilon(a, b) = \mathrm{sg}(K(d(Sa, b))).$$

Coverage and separation prove $K(n) = 0 \leftrightarrow n = 1$ and $\mathrm{sg}(n) = 0 \leftrightarrow n = 0$. Therefore

$$\epsilon(a, b) = 0 \leftrightarrow a = b, \qquad \epsilon(a, b) = 0 \vee \epsilon(a, b) = 1.$$

This is a whole-domain theorem, not an experiment on standard numerals.

Every fixed quantifier-free formula has a Boolean characteristic term:

$$\begin{aligned} b_{s=t} &= \epsilon(s, t), & b_{\bot} &= 1, \\ b_{A \wedge B} &= C(b_A, b_B, 1), & b_{A \vee B} &= C(b_A, 0, b_B), & b_{A \to B} &= C(b_A, b_B, 0). \end{aligned}$$

External induction on formula syntax, using the two Boolean cases, proves $b_A = 0 \leftrightarrow A$. In the implication case only the derived decidability of the quantifier-free antecedent is used. Equation induction on $b_{A(n)} = 0$ gives precisely the quantifier-free induction schema of iPRA, with the same side assumptions.

6.2 The two whole-domain interpretations

Let T_R be ordinary intuitionistic first-order logic with equality, the recursor computations, coverage, separation, and contextual equation induction. Interpret each fixed body $h(i, u, \vec{p})$ in iPRA by its primitive-recursive description, and interpret its recursor by

$$r_h(0, a, \vec{p}) = Sa, \qquad r_h(Sn, a, \vec{p}) = h(n, r_h(n, a, \vec{p}), \vec{p}).$$

This gives $\alpha : T_R \to \mathrm{iPRA}$. Canonical zero nodes are translated by successor. Coverage in iPRA follows by quantifier-free induction on $x = 0 \vee x = S(p(x))$, where p is its standard primitive-recursive predecessor: zero takes the left disjunct and each successor takes the right disjunct by $p(Sn) = n$. Existential introduction then gives the stated coverage axiom. All other axioms translate immediately.

Conversely compile each fixed primitive-recursive description by the displayed Rec to obtain $\beta : \mathrm{iPRA} \to T_R$. Its defining equations, successor axioms, and complete quantifier-free induction have all been proved above. Both maps leave the numerical domain and actual equality unchanged.

Pointwise equal fixed update bodies and equal seeds give equal recursors: induct on a fresh counter in this one equation. The zero case uses successor congruence on the seeds. The successor case replaces the preceding state by the induction equation and then uses the pointwise update equality. This body-extensionality lemma handles different syntactic descriptions of extensionally equal update functions; they need not be identified as symbols.

For an R term, external induction on its finite syntax identifies its children and update body with their translated composites. One equation induction compares the original and compiled trajectories, with identical seed Sa and pointwise equal updates. Thus $T_R \vdash t^{\alpha\beta} = t$. For a primitive-recursive description, external induction on that description does the converse. A recursion case again uses one equation induction to identify two trajectories with equal bases and steps. Hence $\mathrm{iPRA} \vdash t^{\beta\alpha} = t$. Induction on ordinary formula syntax lifts these equalities to both formula round trips. The internally definable numerical composite isomorphisms have graph $x = y$.

6.3 Normalization does not hide an arithmetic assumption

Canonical normalization recursively normalizes all children, and replaces $R(0, a; i, u.h)$ by $R(0, a; i, u.0)$. It is a terminating finite-tree traversal; it performs no successor-counter unfolding. It is idempotent, scope preserving, and obeys

$$N(t\sigma) = N(N(t)N(\sigma)).$$

Once a counter has become literal zero, subsequent substitution cannot make its erased body relevant again. This gives the substitution identity by structural induction. Under binders the usual lifting conventions are used.

In the raw ordinary theory, $N(t) = t$ is proved by external term induction. Use equality congruence for the counter and initializer, the preceding body-extensionality lemma for a changed update body, and the raw zero equation at an erased body. Thus normalization is justified even beneath arbitrary enclosing recursors. The argument uses already available equation induction in the ordinary theory, not an unproved equality principle in the new checker. Its proof-theoretic converse is supplied by the recovered computation, equality, and equation-induction rules of the canonical calculus.

These are written proofs. The existing Lean formalization of the scalar calculus does not by itself certify this different counter-aware kernel, its canonical substitution code, or a new checker implementation.

7 The complete checker and its correctness boundary

7.1 Concrete scope and binder representation

The JSON syntax uses integer 0 for numerical zero, positive integers for one-based bound indices, and negative integers for free parameter levels. At scope n and binding depth d, a numerical leaf is valid exactly when it is an actual Python integer and $-n \le i \le d$. Booleans are excluded; Python's subclass relationship between `bool` and `int` is not used as a typing judgment.

In an R body the state has index 1 and the counter index 2. In a D matrix the universal coordinate y has index 1 and the existential coordinate x index 2. A deeper two-variable binder adds 2 to references to outer bound variables. Negative free levels do not shift when a new parameter is opened.

The public input has exactly the keys `goal` and `proof`. The goal is a closed canonical Eq/D formula. At the root the clause context is empty. Each Open or Close branch introduces exactly one fresh free parameter and appends exactly one clause; CE and Use do neither. Induction on checker calls therefore proves

$$\text{scope} = \text{length}(\Gamma).$$

This is an implementation invariant for certificates built from the empty root, not a mathematical assertion that every abstract clause context has as many free variables as clauses.

7.2 The shared walker

The function `walk` validates the current node's sort and arity, recursively visits every child with the uniform depth increment, applies a supplied operation to numerical leaves, and performs canonical body erasure at a literal-zero recursor counter. It has two uses:

1. `syntax` supplies the scope-checking identity operation on leaves, and requires the returned canonical tree to equal the input tree. This checks grammar, scope, and canonical form together.

2. `opening` supplies the index operation

$$i \longmapsto \begin{cases} t, & i = d+1, \\ i-1, & i > d+1, \\ i, & i < d+1. \end{cases}$$

   The inserted value has no exposed positive indices at its insertion boundary; its own internal binders remain internal. The surrounding depth d is threaded through the same walker.

In CE the implementation first opens the state slot with the canonical term $R(t, a; h)$, then opens the former counter slot with t. This equals the displayed simultaneous substitution. Opening an R or D body under a nested binder adds two to the protected depth, so no counter or state reference is captured.

Lemma 7.1 (Validation and opening invariant)**.** *For finite parsed JSON trees, successful* ***`syntax`*** *is equivalent to membership in the stated scoped canonical grammar. On valid inputs with a canonical insertion term valid at ambient binding depth zero,* ***`opening`*** *computes exactly canonical capture-avoiding opening. Its output agrees with the inherited counter checker's opening operation.*

Proof. Proceed by tree induction. The integer clause checks the precise sort and scope interval. At a list node the allowed arity determines whether the children are numerical or formula children. The final child of an R or D receives two additional protected indices; the equation children receive none. Thus each recursive call has exactly the grammar's required scope. Every child is traversed before the zero-counter erasure. For the syntax use, equality with the returned tree excludes every noncanonical zero-counter body. For the opening use, the three index cases are ordinary one-slot opening, and the same final erasure occurs in both implementations. This proves the two assertions together. □

The checker's CE case performs an opening computation before its two premise checks finish. This does not bypass validation: proving the major goal $s = t$ validates both endpoints, and proving the minor goal $R(Ss, a; h) = r$ validates the seed and the *original* update body. Its outer counter is syntactically a successor, so that original body cannot be erased from the minor before validation.

7.3 Proof certificates

A well-formed reflexive goal is accepted immediately and its unused payload is ignored. This is the existing incoming scalar checker's policy. Every other certificate is one of the following arrays:

Array	Meaning
`["v",s,t,a,h,p,q]`	CE, with the two recursively checked premises $s = t$ and $R(Ss, a; h) =$ goal.left.
`["u",j,t,p]`	Use clause number j at target t; p proves its antecedent.
`["e",B,p,q]`	Open a proof p of DB; q proves the original goal in the extended clause scope.
`["i",t,p]`	Close the goal DB at witness t; p proves its required successor instance in the extended scope.

Array lengths and instruction tags are checked by Python structural pattern matching; clause indices are zero-based, oldest first, and must be actual nonnegative in-range integers. All dynamically formed premise goals are checked on entry. The standard JSON parser's usual last-value behavior for duplicate textual object keys is retained; this creates no unvalidated logical field. Physical memory, running time, and Python recursion limits apply.

Theorem 7.2 (Checker correspondence)**.** *Ignoring physical resource bounds, a closed canonical formula has a certificate accepted by the selected checker if and only if it is a theorem of $\mathcal{C}_R$. More precisely, accepted certificates and certificates for the inherited forward counter calculus have effective translations in both directions with literally identical final formulas.*

Proof. Use the scope invariant and Lemma 7.1. An accepted reflexive goal maps to an explicit inherited Refl node. Each other accepted branch maps to its displayed CE, Use, Open, or Close inference after recursively translating its premises. This proves soundness by induction on the accepted call tree. In the other direction, induct on the finite forward derivation. Insert its derived endpoints and its minor's seed/body as the goal checker's CE hints; copy the clause index and witness for Use; copy the matrix for Open and witness for Close. The same opening invariant identifies every required premise goal. A forward Refl becomes any payload at the corresponding reflexive target. Thus each construction is finite and preserves the end formula exactly. The producer scripts implement these inductions and check both endpoints; they are not imported by the trusted checker. □

Combining Theorems 1.1 and 7.2 gives a complete certificate presentation of iPRA under the explicit logical and numerical translations. No external evaluator, theorem database, proof assistant, or certificate producer is a runtime dependency of the trusted module. The Python interpreter and standard JSON parser are the same ambient runtime assumptions used for the controls.

8 Executed verification and reproducibility

The tests are finite implementation evidence. Universal soundness, completeness, and bi-interpretability rest on the mathematical arguments.

Verification	Observed result
Inherited closed theorem certificates	37 accepted
Literal-goal bidirectional round trips	37 passed
Checked translation directions	74 passed
Adversarial closed certificates	56 rejected
Public CLI executions, normal Python and `-O`	186 passed
Independent grammar comparisons	16,464 agreements
Independent numerical opening identities	900 agreements
Independent counter-evaluation semantic identities	304 agreements
Generated open template-transport derivations	160 accepted
Explicit erased-body boundary cases	12 rejected
Skipped independent cases / mismatches	0 / 0

The positive theorem corpus exercises an induction hypothesis that is actually used, nested term congruence, compound D introduction, successor witnesses, and nonliteral witnesses provably equal to zero. The independent numeric interpreter uses bounded concrete values solely to test the code, not to infer a universal theorem. Its update bodies distinguish state index 1 from counter index 2 and include nested binders. The rejection sets include Boolean clause indices, scope escapes, extra instruction fields, malformed sorts, noncanonical or malformed erased bodies, and forced false closed equations. The categories are reported separately; they are not counted as independent mathematical discoveries.

From the extracted package directory run:

```
python3 reproduce.py
python3 code/checker.py < examples/positive/used_induction_three.json
python3 code/checker.py < examples/negative/false_equation.json
```

The positive command prints `ACCEPT` with status zero. The negative command prints `REJECT` with nonzero status. Use `python3 reproduce.py -pdf` to rebuild the standalone paper. The checker requires Python 3.10 or later for pattern matching; exact reported measurements use Python 3.12.14. Verification and measurements need only the Python standard library. PDF rebuilding needs pdfLaTeX and the standard packages in the preamble.

The archive includes the complete readable checker, its normalized and gzip-9 forms, both comparison controls, proof translators, source and target fixtures, independent tests, measured results, explicit provenance, and a SHA-256 manifest. The report's main `.tex` file is standalone: its checker listing and every mathematical section are embedded.

9 Limits and discarded directions

This result is a modest checker simplification, not a significant new arithmetic foundation. It retains five primitive inference families and the inherited witness-sensitive case/progress distinction. The code reductions are real in all three primary source coordinates, but they do not establish an absolute minimum and do not surpass the repository's compact encodings. Compression against the exact incoming scalar program gets worse, as reported.

A literal Eq/D formula can be flattened to a triple (k, s, t) because it is a unary D chain ending in an equation. That change is mathematically exact, but the tested straightforward implementation grew: rank checking and endpoint opening outweighed the saved recursive formula traversal. It is not selected or claimed as an improvement here.

A reverse-only parameter-substitution rule was also considered. Its soundness was immediate, but the intended fresh-alias completeness argument did not work: once a goal no longer mentions the alias, the rule can require that same goal as its premise. It is excluded from this construction.

The existing Lean scalar development proves its own detailed comparison with iPRA [7]. It was inspected as source evidence, not rebuilt as part of this deliverable. The counter-aware kernel and its Python implementation have not received a new Lean or Rocq verification. A useful next formalization target is the sorted walker invariant together with the inherited counter-to-iPRA interpretation, rather than another unsupported reduction in the number of named rules.

References

[1] V. Reshetnikov's ProveIt repository, *Concise presentations of intuitionistic primitive recursive arithmetic*. Pinned snapshot `0ebd0c6b358f38bb0ad2883faf775421599c8aae`. ConciseIPRA subtree and README.

[2] ConciseIPRA, *counter_evaluation*, especially `code/counter_eval.py` and the report's counter-aware arithmetic bridge. Pinned research package. The exact forward checker and source theorem fixtures are included here.

[3] ConciseIPRA incoming package, *Five-rule iPRA with a single evaluation schema*, `ipra_clausal_single_evaluation_2026-11-11.zip`. Pinned incoming archive. Its exact `code/checker.py` is the 2,283-byte control.

[4] ConciseIPRA incoming package, *iPRA with uniform clause use*, `ipra_uniform_clauses.zip`, in the same pinned incoming directory. It explicitly accounts for a retained reflexivity axiom and four instruction families; no such axiom-count claim is made here.

[5] ConciseIPRA incoming package, *Deterministic specialization for iPRA*, `ipra_deterministic_specialization_2026-11-11.zip`, same pinned incoming directory. Includes the context-length scope invariant and a separate compact native validator.

[6] ConciseIPRA, *certified_zero/reference*: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`. Attributed foundational proofs. Copies supplied by the incoming baseline are retained with the MIT No Attribution license; the report states the shorter direct counter compiler explicitly.

[7] ConciseIPRA Lean sources, notably `NativeBiInterpretation.lean`, `NativeEval.lean`, and `NativeSix.lean`. Pinned formalization sources. The native scalar results do not automatically certify this different counter-aware checker.

A The complete selected checker

The following listing is the whole trusted Python module, including its command-line envelope. It is reproduced without omitting helpers.

```
"""Canonical counter-recursion and Eq/D clause logic, with one shared walker.

Numerical terms: 0, positive bound indices, negative free levels, and
[n, a, h] (R). R binds counter and state in h; both R and D bind two
variables in their final child. R(0,a,h) is canonical only with h=0,
so S(a) is [0,a,0]. Formulas: [s,t] (Eq), [B] (D).
A validated reflexive goal closes regardless of its unused proof payload.
"""
import json
import sys


def require(condition):
    if not condition:
        raise ValueError('invalid certificate')


def walk(a, f, depth=0, form=False):
    if type(a) is int:
        require(not form)
        return f(a, depth)
    require(type(a) is list)
    size = len(a)
    require(size in ((1, 2) if form else (3,)))
    result = [walk(t, f, depth + 2 * (size != 2 and i == size - 1),
                   size == 1)
              for i, t in enumerate(a)]
    return [0, result[1], 0] if size == 3 and result[0] == 0 else result
```

```
def syntax(a, scope, form=False):
    def variable(i, depth):
        require(-scope <= i <= depth)
        return i
    require(walk(a, variable, form=form) == a)


def opening(a, value, depth=0):
    return walk(a, lambda i, d: value if i == d + 1 else i - (i > d + 1), depth, type(a) is list and len(a) < 3)


def prove(a, p, g):
    n = len(g)
    syntax(a, n, True)
    if len(a) == 2 and a[0] == a[1]:
        return
    match p:
        case ['v', s, t, b, h, major, minor]:
            require(len(a) == 2 and a[1] == opening(opening(h, [t, b, 0 if t == 0 else h]), t))
            prove([s, t], major, g)
            prove([[[0, s, 0], b, h], a[0]], minor, g)
        case ['u', index, t, premise]:
            require(type(index) is int and 0 <= index < len(g))
            syntax(t, n)
            antecedent, body = g[index]
            require(a == opening(body, t))
            prove(antecedent, premise, g)
        case ['e', body, major, branch]:
            prove([body], major, g)
            body = opening(body, -n - 1, 1)
            prove(a, branch, g + [(opening(body, 0), body)])
        case ['i', t, branch]:
            require(len(a) == 1)
            syntax(t, n)
            body = opening(a[0], t, 1)
            hypothesis = opening(body, -n - 1 if len(a[0]) == 2 and t == 0 else 0)
            prove(opening(body, [0, -n - 1, 0]), branch, g + [([0, 0], hypothesis)])
        case _:
            raise ValueError('invalid proof')


def check(c):
    require(type(c) is dict and set(c) == {'goal', 'proof'})
    prove(c['goal'], c['proof'], [])


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```